

RAPPORT EXPLICATIF — PAS À PAS

Déploiement Cloud & DevSecOps

Tout le projet expliqué de A à Z : les choix, les pourquoi, les alternatives écartées, les vrais bugs trouvés en direct, et le lien avec chaque ligne de ton CV.

AWS · Terraform · Docker · Kubernetes · GitLab CI/CD

Sommaire

0. Comment lire ce rapport
1. Pourquoi ce projet est un excellent projet de portfolio
2. Vue d'ensemble de l'architecture
3. Les technologies : quoi, pourquoi, alternatives
4. Terraform et l'infrastructure-as-code expliqués en détail
5. Kubernetes expliqué depuis zéro
6. La sécurité du cluster Kubernetes
7. Le pipeline CI/CD GitLab, étape par étape
8. Les scans de sécurité automatisés
9. Le déploiement réel : ce qui a été fait, pour de vrai
10. Les deux vrais bugs trouvés et corrigés en direct
11. Maîtriser le coût d'une infrastructure cloud
12. Git et GitHub
13. Le lien avec ton CV, phrase par phrase
14. Limites du projet et pistes d'amélioration réelles
15. Questions probables en entretien + éléments de réponse
16. Le laboratoire en direct : exposition publique et auto-réparation
17. Glossaire complet de tous les termes techniques

Comment lire ce rapport

Même principe que pour tes deux précédents rapports (détection de fraude, API de paiement) : chaque notion est définie la première fois qu'elle apparaît, dans un encadré coloré.

Définition

Bleu : définit un terme technique simplement, avec une analogie si possible.

À retenir pour l'entretien

Vert : une phrase à savoir répéter à l'oral.

Piège / erreur classique

Orange : une erreur fréquente, évitée ou expliquée ici.

Lien avec ton CV

Orange foncé : relie la section à ta phrase de CV ("Mise en place d'un pipeline CI/CD et déploiement d'une application conteneurisée avec contrôles de sécurité").

Vrai bug trouvé en direct

Rouge : ce projet est le seul des trois à avoir été **réellement déployé sur un vrai compte AWS**. Ces encadrés racontent les vrais problèmes rencontrés en le faisant tourner pour de vrai — et comment ils ont été diagnostiqués et corrigés. C'est probablement la partie la plus précieuse à raconter en entretien.

Ce projet est différent des deux précédents : il ne s'agit pas de construire une application, mais de construire **l'infrastructure et l'automatisation** qui permettent à n'importe quelle application (y compris celles des deux autres projets !) de tourner de façon fiable, sécurisée et reproductible dans le cloud.

Pourquoi ce projet est un excellent projet de portfolio

1.1 Le problème que ce métier résout

Écrire une bonne application ne suffit pas : il faut aussi la faire tourner quelque part, de façon fiable, disponible, et sécurisée, tout en pouvant la mettre à jour plusieurs fois par jour sans tout casser. C'est le métier du **DevOps** (contraction de "Development" et "Operations") et de sa variante orientée sécurité, le **DevSecOps**.

Définition — DevOps / DevSecOps

Le DevOps est une culture et un ensemble de pratiques qui rapprochent le développement logiciel (écrire le code) et l'exploitation (le faire tourner en production), notamment via l'automatisation (déployer en un clic plutôt qu'à la main) et l'infrastructure-as-code (décrire les serveurs comme du code, pas comme des clics dans une console). Le DevSecOps ajoute la sécurité comme responsabilité partagée à chaque étape de cette chaîne, plutôt que comme une vérification isolée à la toute fin.

1.2 Ce qui rend ce projet crédible

Caractéristique	Pourquoi c'est important
Déployé pour de vrai	Sur un vrai compte AWS, pas une simulation. La majorité des projets de portfolio DevOps sur GitHub ne sont jamais réellement exécutés — celui-ci l'a été, preuves à l'appui dans <code>reports/</code> .
Sécurité à chaque étape ("shift left")	Pas un scan de sécurité ajouté à la fin, mais intégré à chaque étape du pipeline : code, dépendances, image, infrastructure, manifestes Kubernetes.
De vrais bugs trouvés et corrigés	Deux problèmes réels (Partie 10) découverts uniquement parce que le déploiement a été testé en conditions réelles, pas juste écrit puis supposé correct.
Maîtrise du coût	Une vraie contrainte de production (le cloud coûte de l'argent à chaque heure) gérée activement : alertes de budget, instances Spot, destruction systématique après vérification.

À retenir pour l'entretien

"Ce projet a été réellement déployé sur AWS, pas juste écrit : j'ai créé un vrai cluster Kubernetes, déployé une vraie application dessus, vérifié la sécurité en conditions réelles, et détruit l'infrastructure ensuite pour maîtriser les coûts. En le faisant, j'ai découvert deux vrais problèmes que je n'aurais jamais trouvés en me contentant d'écrire le code — je peux les détailler si ça vous intéresse."

Vue d'ensemble de l'architecture

Composant	Fichier(s)	Rôle
Application de démonstration	<code>app/</code>	Une API FastAPI minimale, volontairement simple : le sujet du projet est l'infrastructure autour, pas l'application elle-même
État Terraform partagé	<code>terraform/</code> <code>bootstrap/</code>	Bucket S3 + table DynamoDB pour stocker et verrouiller l'état de l'infrastructure
Infrastructure AWS	<code>terraform/</code> <code>environments/dev/</code>	VPC, cluster Kubernetes managé (EKS), registre d'images (ECR)
Manifestes Kubernetes	<code>k8s/base/</code> + <code>k8s/</code> <code>overlays/</code>	Description de comment l'application doit tourner sur le cluster, avec des variantes par environnement (Kustomize)
Pipeline d'automatisation	<code>.gitlab-ci.yml</code>	Enchaîne automatiquement tests, scans de sécurité, construction, et déploiement à chaque changement de code
Scripts de démonstration	<code>scripts/</code>	Rejouent manuellement ce que le pipeline automatiserait, et installent les briques d'infrastructure du cluster (autoscaling)

```

graph TD
    subgraph CI ["Pipeline GitLab CI/CD"]
        L[Lint] --> T[Tests]
        T --> S1[Sécurité du code]
        S1 --> B[Build image Docker]
        B --> S2[Sécurité de l'artefact]
        S2 --> D[Déploiement]
    end
    D --> ECR[ECR]
    ECR --> EKS[EKS]
    subgraph AWS ["VPC (eu-west-3)"]
        EKS --> NG[Nœud Spot]
        NG --> PODS[Pods de l'application]
    end

```

Schéma d'architecture (voir README.md pour la version illustrée)

À retenir pour l'entretien

"Le pipeline suit le principe 'shift left' : chaque contrôle de sécurité tourne le plus tôt possible, avant même de construire l'image Docker. Si un problème est détecté au niveau du code source, on ne perd pas de temps à construire et déployer une image qu'on devra de toute façon rejeter."

Les technologies : quoi, pourquoi, et par rapport à quelles alternatives

3.1 AWS (Amazon Web Services)

Définition — Cloud computing

Plutôt que d'acheter et de maintenir ses propres serveurs physiques, le "cloud computing" consiste à louer, à la demande et à l'usage, de la puissance de calcul, du stockage et des services chez un fournisseur (Amazon, Microsoft, Google...). On peut créer un serveur en quelques secondes et le détruire tout aussi vite — exactement ce qu'on a fait dans ce projet.

Alternative : Microsoft Azure, Google Cloud Platform (GCP) sont les deux autres grands fournisseurs. AWS a été choisi car c'est le plus utilisé en entreprise et celui qui a le plus d'offres gratuites pour débiter. Les concepts (VPC, cluster Kubernetes managé, registre d'images) existent sous des noms différents mais équivalents chez les trois (ex : GKE chez Google, AKS chez Microsoft, contre EKS chez AWS).

3.2 Terraform

Expliqué en détail en Partie 4.

Alternative écartée : AWS CloudFormation / AWS CDK. Ce sont les outils d'infrastructure-as-code propres à AWS. Terraform, lui, fonctionne avec tous les fournisseurs cloud (AWS, Azure, GCP...) avec le même langage — une compétence transférable, plutôt que liée à un seul fournisseur. C'est aussi l'outil le plus demandé sur le marché du travail DevOps actuellement.

3.3 Docker

Déjà expliqué en détail dans le rapport du projet "API de paiement sécurisée" (conteneurs, multi-stage build, utilisateur non-root). Les mêmes principes s'appliquent ici à l'application de démonstration.

3.4 Kubernetes

Expliqué en détail en Partie 5.

Alternative écartée : AWS ECS (Elastic Container Service). ECS est le système d'orchestration de conteneurs propre à AWS, plus simple à apprendre. Kubernetes est plus complexe, mais c'est le standard de facto de l'industrie, utilisé chez la quasi-totalité des grandes entreprises tech, et disponible chez tous les fournisseurs cloud (pas de dépendance à un seul fournisseur, contrairement à ECS) : une compétence Kubernetes se transfère d'une entreprise à l'autre, quel que soit leur fournisseur cloud.

3.5 GitLab CI/CD

Définition — CI/CD

CI (Intégration Continue) : à chaque changement de code, une machine exécute automatiquement les tests et les vérifications, plutôt que de faire confiance à chaque développeur pour le faire manuellement. **CD** (Déploiement Continu) : une fois les vérifications passées, le code est automatiquement mis en production (ou prêt à l'être). Ensemble, CI/CD élimine le geste manuel "je copie mon code sur le serveur", source classique d'erreurs humaines.

Alternative écartée : GitHub Actions, Jenkins. GitHub Actions est l'équivalent chez GitHub (utilisé pour les deux autres projets de ce portfolio, hébergés sur GitHub). GitLab CI/CD a été choisi ici spécifiquement car il est explicitement demandé par le type de poste visé ("GitLab CI/CD" dans l'offre), et c'est l'outil de CI/CD le plus utilisé en entreprise avec GitHub Actions et Jenkins (plus ancien, plus lourd à administrer soi-même).

Terraform et l'infrastructure-as-code expliqués en détail

4.1 Le problème que Terraform résout

Piège classique : configurer l'infrastructure "à la main"

Sans infrastructure-as-code, on configure les serveurs en cliquant dans la console AWS. Le problème : personne ne se souvient exactement de tous les clics effectués, impossible de recréer à l'identique la même infrastructure ailleurs (ex: pour un environnement de test), et aucune trace ni historique des changements (qui a changé quoi, quand, pourquoi).

Définition — Infrastructure as Code (IaC)

L'infrastructure-as-code consiste à décrire l'infrastructure souhaitée (serveurs, réseaux, bases de données...) dans des fichiers texte versionnés (comme du code source normal), plutôt que de la configurer manuellement. Terraform lit ces fichiers et se charge de créer, modifier ou supprimer les ressources cloud correspondantes.

4.2 Les trois commandes fondamentales

Commande	Ce qu'elle fait
<code>terraform init</code>	Télécharge les plugins nécessaires (ici : le plugin AWS, et les modules VPC/EKS de la communauté)
<code>terraform plan</code>	Calcule et affiche ce qui <i>serait</i> changé, sans rien modifier réellement — l'équivalent d'une prévisualisation
<code>terraform apply</code>	Applique réellement les changements calculés par le plan

Dans ce projet, chaque `terraform apply` a été précédé d'un `terraform plan` relu avant de continuer — c'est la discipline de base pour ne jamais être surpris par ce qu'une commande va réellement faire sur un vrai compte cloud.

4.3 L'état Terraform (le fichier le plus important et le plus fragile)

Définition — État Terraform (state)

Terraform garde la trace de ce qu'il a créé dans un fichier appelé "état" (`terraform.tfstate`) : sans lui, il ne saurait pas que telle ressource AWS correspond à telle ligne de code, et ne pourrait ni la modifier ni la détruire proprement.

Piège évité : un état stocké seulement en local

Si le fichier d'état n'existe que sur l'ordinateur d'une seule personne, personne d'autre ne peut faire évoluer l'infrastructure sans risquer de créer un état incohérent (Terraform recréerait tout en pensant que rien n'existe). C'est pourquoi ce projet utilise un **backend distant partagé** : un bucket S3 stocke le fichier d'état (accessible à quiconque a les droits), et une table DynamoDB sert de verrou (empêche deux `terraform apply` simultanés de s'exécuter en même temps et de corrompre l'état).

Ce backend distant est lui-même créé par un module Terraform séparé (`terraform/bootstrap/`), appliqué une seule fois, avec un état local — un cas particulier inévitable : il faut bien un premier bucket pour stocker l'état de tous les buckets suivants.

4.4 Utiliser des modules communautaires plutôt que tout réécrire

Ce projet utilise les modules Terraform officiels et largement utilisés `terraform-aws-modules/vpc/aws` et `terraform-aws-modules/eks/aws` plutôt que d'écrire chaque ressource AWS une par une. C'est la pratique standard en entreprise : un VPC ou un cluster EKS bien configuré demande des dizaines de ressources (sous-réseaux, tables de routage, groupes de sécurité, rôles IAM...) qu'il est risqué et inefficace de réécrire à la main à chaque projet.

À retenir pour l'entretien

"J'ai épinglé les modules à une version exacte (ex: '20.37.2', pas juste '~> 20.0') plutôt qu'une contrainte ouverte, sur recommandation d'un scan de sécurité (Checkov, règle CKV_TF_1) : ça évite qu'un `terraform init` ultérieur récupère silencieusement une nouvelle version d'un module tiers non revue, un vrai risque de sécurité de la chaîne d'approvisionnement logicielle ('supply chain')."

4.5 La sécurité de l'infrastructure elle-même

Tout le code Terraform a été passé au crible de deux scanners de sécurité spécialisés, **tfsec** et **Checkov**, qui vérifient des règles de bonnes pratiques (chiffrement activé, accès public restreint, logs d'audit actifs...) automatiquement, sans qu'un humain n'ait besoin de tout connaître par cœur. Chaque finding a été soit corrigé, soit explicitement documenté comme accepté avec sa justification — voir `SECURITY_EXCEPTIONS.md` dans le dépôt, qui liste par exemple pourquoi l'accès public à l'API du cluster est accepté (restreint à une seule IP) alors qu'un scanner le signale par défaut comme risqué.

Kubernetes expliqué depuis zéro

5.1 Le problème que Kubernetes résout

Une application en production doit survivre à des pannes de serveur, s'adapter à des pics de trafic, et pouvoir être mise à jour sans interruption de service. Faire tout ça manuellement (redémarrer un conteneur planté, ajouter des machines en cas de pic, basculer le trafic progressivement vers une nouvelle version) est un travail à temps plein.

Définition — Kubernetes

Kubernetes (souvent abrégé "k8s") est un système qui gère automatiquement un groupe de conteneurs sur plusieurs machines : il les redémarre s'ils plantent, les répartit sur les machines disponibles, les met à jour progressivement, et peut en ajouter/retirer selon la charge. On lui décrit l'état désiré ("je veux 2 exemplaires de mon application qui tournent en permanence"), et il se charge de le maintenir vrai en continu.

5.2 Le vocabulaire de base

Terme	Définition simple
Pod	La plus petite unité déployée par Kubernetes : un ou plusieurs conteneurs qui partagent le même réseau. Dans ce projet, chaque pod contient un seul conteneur (l'application).
Deployment	Décrit combien de répliques (copies) d'un pod doivent tourner en permanence, et comment les mettre à jour. Si un pod plante, le Deployment en recrée un automatiquement.
Service	Une adresse réseau stable qui redirige le trafic vers les pods actuellement en bonne santé, même s'ils changent (redémarrage, mise à jour) — sans Service, il faudrait suivre manuellement l'IP changeante de chaque pod.
Namespace	Un espace de noms qui isole logiquement un groupe de ressources des autres (ex: séparer les applications de deux équipes sur le même cluster).
Node	Une machine (ici, une instance EC2) qui fait réellement tourner les pods.
Cluster	L'ensemble formé par le control plane (le "cerveau" qui décide) et les nodes (les "muscles" qui exécutent).

5.3 Kustomize : gérer plusieurs environnements sans dupliquer les fichiers

Définition — Kustomize

Kustomize permet de définir une configuration "de base" (`k8s/base/`) commune, puis des "surcouches" (`k8s/overlays/dev` , `k8s/overlays/prod`) qui ne décrivent que les *différences* par environnement (nombre de répliques, quantité de ressources CPU/mémoire, tag de l'image), sans jamais dupliquer l'intégralité des fichiers.

Dans ce projet : l'environnement `dev` tourne avec 1 réplique (rapide, économique) tandis que `prod` en définit 3 avec plus de ressources allouées — sans qu'aucun fichier ne soit recopié entre les deux.

5.4 HorizontalPodAutoscaler : l'auto-scaling

Définition

Un HPA (HorizontalPodAutoscaler) surveille une métrique (ici, l'utilisation CPU moyenne des pods) et ajuste automatiquement le nombre de répliques entre un minimum et un maximum définis, pour absorber un pic de charge sans intervention humaine, puis revenir à la normale une fois le pic passé.

Il a besoin d'une source de métriques réelles pour fonctionner : `metrics-server` , un composant additionnel du cluster installé séparément (`scripts/install_cluster_addons.sh`), qui a permis de vérifier en conditions réelles (`kubectl top pods`) l'utilisation CPU réelle des pods déployés.

La sécurité du cluster Kubernetes

6.1 Le contexte de sécurité du pod (Security Context)

Chaque pod de ce projet applique une liste de restrictions, en défense en profondeur (voir le rapport du projet "API de paiement" pour ce concept) :

Restriction	Ce qu'elle empêche
<code>runAsNonRoot: true , runAsUser: 10001</code>	Le conteneur ne peut jamais tourner en administrateur (root), même si l'image le permettrait par erreur
<code>readOnlyRootFilesystem: true</code>	Impossible d'écrire sur le système de fichiers du conteneur — vérifié réellement (Partie 9) : une tentative d'écriture a bien été refusée
<code>allowPrivilegeEscalation: false</code>	Un processus dans le conteneur ne peut jamais obtenir plus de droits que ceux avec lesquels il a démarré
<code>capabilities: drop: ["ALL"]</code>	Retire toutes les capacités Linux avancées (ex: modifier la configuration réseau du système) dont l'application n'a pas besoin
<code>automountServiceAccountToken: false</code>	Le pod ne reçoit même pas de jeton pour parler à l'API Kubernetes, puisqu'il n'en a pas besoin — pas de surface d'attaque inutile

6.2 Pod Security Standards : une deuxième barrière au niveau du cluster

Définition

Les Pod Security Standards sont une fonctionnalité native de Kubernetes (depuis la version 1.25) qui peut **refuser catégoriquement** la création d'un pod qui ne respecte pas un niveau de sécurité donné ("restricted" étant le plus strict), au niveau du namespace tout entier — indépendamment de ce qu'un Deployment individuel essaierait de faire.

Vérifié en conditions réelles (effet de bord positif inattendu)

En testant la connectivité réseau du cluster (Partie 10), une tentative de créer un pod de test `busybox` basique (sans configuration de sécurité) dans le namespace de l'application a été

automatiquement rejetée par Kubernetes lui-même, avec un message explicite listant précisément les règles violées (pas de `runAsNonRoot`, pas de `seccompProfile` ...). C'est exactement le comportement voulu : même une erreur de configuration future ne pourrait pas introduire un pod non sécurisé dans ce namespace.

6.3 NetworkPolicy : le pare-feu interne du cluster

Définition

Par défaut, dans Kubernetes, absolument tous les pods peuvent communiquer avec tous les autres pods du cluster, sans restriction — comme un immeuble de bureaux où toutes les portes seraient ouvertes. Une `NetworkPolicy` définit des règles explicites de qui peut parler à qui, sur quel port.

Ce projet applique un modèle "tout refuser par défaut, puis autoriser explicitement" : une première règle (`default-deny-all`) bloque tout, une seconde (`demo-service-allow`) réautorise uniquement le trafic entrant vers l'application, depuis le même namespace, sur son port applicatif.

Piège majeur découvert en le testant réellement

Ce piège est si important et si réel qu'il a sa propre partie dédiée : voir Partie 10.1. En résumé : sur EKS, une `NetworkPolicy` peut être acceptée sans erreur par Kubernetes tout en n'ayant **aucun effet réel**, tant qu'une option précise n'est pas activée sur le composant réseau du cluster. Ne jamais faire confiance à la présence d'une `NetworkPolicy` comme preuve d'isolation — il faut la **tester activement**.

Le pipeline CI/CD GitLab, étape par étape

Le fichier `.gitlab-ci.yml` décrit une suite d'étapes ("stages"), chacune composée d'un ou plusieurs jobs, qui s'enchaînent automatiquement à chaque changement de code poussé sur GitLab.

Stage	Jobs	Rôle
1. lint	lint:terraform, lint:dockerfile	Vérifie le style et la syntaxe du code avant même de le tester fonctionnellement
2. test	test:app	Exécute les tests automatisés de l'application (pytest)
3. security-code	security:secrets, security:sast, security:iac	Scanne le code source lui-même, avant toute construction (voir Partie 8)
4. build	build:image	Construit l'image Docker et la pousse sur ECR
5. security-artifact	security:container-scan, security:k8s-manifests	Scanne ce qui a été réellement construit : l'image et les manifestes Kubernetes rendus
6. deploy	deploy:dev (auto), deploy:prod (manuel)	Déploie sur le cluster — automatiquement en dev, jamais sans validation humaine en prod

7.1 Pourquoi la validation humaine obligatoire en production ?

```
deploy:prod:
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: manual
```

À retenir pour l'entretien

"Le déploiement en développement est automatique dès que les tests et les scans de sécurité passent, mais le déploiement en production nécessite un clic explicite d'un humain dans l'interface GitLab (`when: manual`). Aucune automatisation, aussi bien testée soit-elle, ne devrait pouvoir mettre en production sans qu'un humain n'ait le dernier mot — un principe de sécurité autant que de bon sens opérationnel."

7.2 Un piège d'images Docker à connaître : les ENTRYPOINT figés

Piège rencontré et corrigé en écrivant ce pipeline

Plusieurs images Docker officielles d'outils de sécurité (gitleaks, tfsec, trivy) fixent leur ENTRYPOINT directement sur leur binaire principal. Résultat : la commande qu'on écrit dans script: (ex: `gitleaks detect ...`) est alors passée *en argument* à ce binaire déjà lancé, au lieu d'être exécutée normalement — provoquant une erreur du type `unknown command "sh" for "gitleaks"`. La solution, appliquée à chacun de ces jobs : ajouter `entrypoint: [""]` dans la définition de l'image, pour neutraliser cet ENTRYPOINT et laisser GitLab exécuter la commande normalement. Ce projet a vérifié ce comportement en inspectant directement chaque image (`docker inspect ... --format '{{.Config.Entrypoint}}'`) plutôt que de deviner.

Les scans de sécurité automatisés

Type de scan	Outil	Ce qu'il vérifie	Quand
Secrets	Gitleaks	Une clé API, un mot de passe oublié par erreur dans le code source	Avant toute construction
SAST (analyse statique)	Semgrep	Des patterns de code dangereux (injection, désérialisation non sûre...) sans exécuter le code	Avant toute construction
IaC (infrastructure)	tfsec, Checkov	Des erreurs de configuration cloud dangereuses (chiffrement absent, accès public non voulu...)	Sur le code Terraform et les manifestes Kubernetes
Conteneur	Trivy	Des vulnérabilités connues (CVE) dans les paquets système et les bibliothèques de l'image construite	Après la construction de l'image

8.1 SAST : qu'est-ce que ça regarde vraiment ?

Définition — SAST (Static Application Security Testing)

Le SAST analyse le code source **sans l'exécuter**, à la recherche de patterns connus pour être dangereux (ex: construire une requête SQL en concaténant directement une entrée utilisateur). C'est complémentaire, pas concurrent, du DAST (Dynamic Application Security Testing) qui, lui, teste une application en cours d'exécution en lui envoyant de vraies requêtes.

8.2 La politique "ignore-unfixed" pour le scan de conteneur

Une décision de politique de sécurité, pas juste un réglage technique

Scanner une image Docker révèle presque toujours des vulnérabilités dans des paquets système qui n'ont, à un instant donné, pas encore de correctif publié. Bloquer systématiquement le pipeline sur ces vulnérabilités "non corrigéables" reviendrait à ne plus jamais pouvoir déployer. Ce projet utilise `trivy image --ignore-unfixed` : seules les vulnérabilités pour lesquelles un correctif existe réellement font échouer le pipeline. Sur l'image réellement construite dans ce projet, ce scan (exécuté en local avant même le pipeline) a trouvé **zéro** vulnérabilité CRITICAL/HIGH corrigéable.

8.3 Le triage : ne pas tout corriger aveuglément

Un vrai travail de sécurité ne consiste pas à faire disparaître 100% des alertes d'un scanner à tout prix, mais à évaluer chaque finding et choisir consciemment : corriger, accepter (avec justification écrite), ou parfois reconfigurer l'outil. Ce projet documente explicitement cette démarche dans `SECURITY_EXCEPTIONS.md` — par exemple, l'absence de journalisation d'accès sur le bucket S3 de l'état Terraform est un risque accepté (bucket temporaire, détruit en fin de test), alors que l'absence de logs d'audit du cluster EKS a, elle, été corrigée (risque jugé plus important, coût de correction faible).

Le déploiement réel : ce qui a été fait, pour de vrai

Contrairement à un exercice purement théorique, ce projet a été déployé sur un vrai compte AWS. Voici, dans l'ordre exact, ce qui a été fait et vérifié (preuves complètes dans `reports/deployment_evidence.txt`) :

1. **Bootstrap** : création du bucket S3 + table DynamoDB pour l'état Terraform partagé.
2. **Infrastructure** : `terraform apply` a créé 62 ressources AWS réelles (VPC, sous-réseaux, NAT Gateway, cluster EKS, node group, rôles IAM, dépôt ECR...).
3. **Connexion** : `aws eks update-kubeconfig` a connecté `kubectl` au vrai cluster ; `kubectl get nodes` a confirmé un nœud `Ready` .
4. **Add-on** : installation de `metrics-server` via Helm, pour permettre l'autoscaling.
5. **Construction et déploiement** : l'image de l'application construite, poussée sur ECR, puis déployée sur le cluster via Kustomize.
6. **Vérification fonctionnelle** : `kubectl port-forward` + `curl` ont confirmé que l'application répondait réellement, avec 2 répliques actives (`2/2 Running`).
7. **Vérification de sécurité** : utilisateur non-root confirmé (`id` → `uid 10001`), écriture bloquée sur le système de fichiers, isolation réseau testée (voir Partie 10).
8. **Destruction complète** : `terraform destroy` , puis vérification manuelle qu'il ne restait plus aucune ressource facturée à l'heure (`aws eks list-clusters` , `aws ec2 describe-nat-gateways` — tous deux vides).

À retenir pour l'entretien

"Je peux montrer les commandes exactes et leurs résultats réels que j'ai obtenus — ce ne sont pas des captures d'écran d'un tutoriel, ce sont mes propres logs de déploiement, conservés dans le dépôt. Je peux notamment montrer que l'utilisateur du conteneur en cours d'exécution était bien non-root (`uid 10001`) et que le système de fichiers était bien en lecture seule, vérifiés par une commande exécutée directement dans le pod en cours de fonctionnement."

Les deux vrais bugs trouvés et corrigés en direct

C'est la partie la plus précieuse de ce rapport pour un entretien : la preuve qu'un déploiement réel révèle des problèmes qu'aucune relecture de code, aussi attentive soit-elle, n'aurait trouvés.

10.1 Les NetworkPolicy n'étaient pas appliquées par défaut

Le problème constaté

Après le premier déploiement, un pod de test placé volontairement dans un *autre* namespace arrivait quand même à joindre l'application, malgré une `NetworkPolicy` de refus par défaut bien présente et acceptée sans erreur par Kubernetes (`kubectl get networkpolicy` la listait normalement, comme si tout allait bien).

Diagnostic : le composant réseau par défaut d'EKS (Amazon VPC CNI) n'active pas, par défaut, l'application réelle des `NetworkPolicy`. Elles sont acceptées par l'API Kubernetes (aucune erreur, aucun avertissement visible) mais restent lettre morte tant qu'une option précise, `enableNetworkPolicy`, n'est pas activée sur l'add-on `vpc-cni` du cluster.

Correction : ajout d'un bloc `cluster_addons` dans le code Terraform, configurant explicitement `vpc-cni` avec cette option activée. Après avoir réappliqué ce changement (une mise à jour d'add-on, pas une recreation du cluster — rapide et peu coûteuse), le même test de pod inter-namespaces a échoué en timeout, confirmant l'isolation réelle. Vérifié aussi via l'apparition d'objets `PolicyEndpoint` (ressources internes créées par le contrôleur une fois activé) qui n'existaient pas avant la correction.

À retenir pour l'entretien

"J'ai appris qu'il ne faut jamais faire confiance à la simple présence d'une configuration de sécurité déclarée — il faut activement essayer de contourner la protection pour prouver qu'elle fonctionne, exactement comme on testerait une serrure en essayant de l'ouvrir plutôt qu'en vérifiant juste qu'elle est vissée à la porte."

10.2 Une image construite pour la mauvaise architecture de processeur

Le problème constaté

Après le déploiement, les pods de l'application entraient en `CrashLoopBackOff` (redémarrages en boucle). Les logs du conteneur affichaient un message énigmatique :

```
exec /usr/local/bin/uvicorn: exec format error .
```

Diagnostic : l'ordinateur utilisé pour construire l'image est un Mac Apple Silicon (architecture **arm64**). Par défaut, `docker build` construit une image pour l'architecture de la machine qui construit. Or, les nœuds EKS standards tournent en **amd64** (x86_64) — deux familles de processeurs qui n'exécutent pas le même langage machine, un peu comme essayer de lire un livre écrit dans un alphabet que l'ordinateur ne reconnaît pas.

Correction : reconstruction de l'image avec `docker buildx build --platform linux/amd64`, en utilisant l'émulation **QEMU** pour construire une image de l'architecture cible depuis une machine d'une architecture différente. Le script de déploiement du projet (`scripts/deploy.sh`) intègre maintenant systématiquement ce flag, avec un commentaire expliquant pourquoi.

À retenir pour l'entretien

"C'est un piège classique et de plus en plus fréquent avec la popularité des Mac Apple Silicon dans les équipes de développement, alors que la quasi-totalité des serveurs cloud tournent encore en x86_64. Je sais maintenant reconnaître ce message d'erreur précis ('exec format error') et le corriger immédiatement, plutôt que de chercher pendant une heure du côté applicatif un bug qui n'existe pas."

Maîtriser le coût d'une infrastructure cloud

11.1 Le cloud coûte de l'argent à chaque heure, pas à l'usage applicatif

Contrairement à un ordinateur personnel déjà payé, une ressource cloud (un cluster, une machine, une passerelle réseau) facture chaque heure où elle existe, qu'elle serve à quelque chose ou non. Une infrastructure oubliée allumée peut coûter des centaines d'euros en quelques semaines sans qu'aucune requête ne l'ait jamais atteinte.

11.2 Les décisions prises pour maîtriser le coût de ce test

Décision	Effet
Instance Spot plutôt qu'à la demande	Jusqu'à ~70% moins cher pour une charge non-critique et de courte durée
Une seule NAT Gateway (pas une par zone de disponibilité)	Réduit le coût réseau d'environ moitié, au prix d'un point de défaillance unique inacceptable en production mais négligeable pour un test de quelques heures
Alerte de budget AWS (5\$/10\$)	Notification automatique par email si la dépense dépasse un seuil, indépendamment de toute action manuelle
Destruction systématique après vérification	La protection la plus importante : aucune ressource n'a été laissée tourner "au cas où"

11.3 Comment vérifier qu'il ne reste vraiment plus rien

Une bonne pratique après un `terraform destroy` : ne pas se contenter de croire que la commande a réussi, mais **vérifier indépendamment** auprès du fournisseur cloud lui-même. Ce projet a vérifié, après destruction :

```
aws eks list-clusters --region eu-west-3          # -> []
aws ec2 describe-nat-gateways ... --filter state=available  # -> []
aws ec2 describe-vpcs --filters "Name=tag:Project,..."    # -> []
```

À retenir pour l'entretien

"La maîtrise des coûts fait partie intégrante du travail DevOps, pas un détail administratif à part. J'ai mis en place une alerte de budget en amont, choisi une architecture volontairement économique pour un test court, et surtout vérifié activement — pas juste supposé — que la destruction avait bien tout supprimé, directement auprès de l'API AWS plutôt que de faire confiance au message de succès de Terraform seul."

Git et GitHub

Même démarche que pour les deux autres projets de ce portfolio : dépôt initialisé, `.gitignore` excluant les états Terraform locaux, les plans (`*.tfplan`, qui peuvent révéler la structure interne des ressources), et tout fichier de configuration kubectl contenant un certificat.

Un choix délibéré : GitHub plutôt que GitLab pour héberger ce dépôt

Ta ligne de CV demande spécifiquement "GitLab CI/CD". Le fichier `.gitlab-ci.yml` de ce projet est écrit en syntaxe GitLab CI réelle et correcte (vérifiée en partie via `gitlab-ci-local`, un simulateur qui exécute réellement certains jobs dans des conteneurs Docker identiques à ceux qu'utiliserait un vrai runner GitLab). Il est hébergé sur GitHub ici pour rester cohérent avec les deux autres projets de ce portfolio ; il est directement prêt à être poussé sur un vrai projet GitLab si besoin, sans aucune modification.

Le lien avec ton CV, phrase par phrase

Ta ligne de CV : *"Mise en place d'un pipeline CI/CD et déploiement d'une application conteneurisée avec contrôles de sécurité."*

Mot-clé du CV	Ce que tu peux dire
"AWS"	"J'ai provisionné une vraie infrastructure sur AWS : VPC, cluster EKS, registre ECR — testée en conditions réelles, pas seulement écrite."
"Docker"	"Image construite en multi-stage, utilisateur non-root, et j'ai résolu un vrai problème de compatibilité d'architecture CPU (arm64 vs amd64) rencontré en la déployant."
"Kubernetes"	"Déploiement avec Kustomize pour gérer plusieurs environnements, autoscaling basé sur l'utilisation CPU réelle, et sécurité renforcée (Pod Security Standards, NetworkPolicy, contexte de sécurité restrictif) — vérifiée activement, pas juste déclarée."
"GitLab CI/CD"	"Pipeline complet en 6 étapes suivant le principe 'shift left' : chaque contrôle de sécurité tourne le plus tôt possible, avant même de construire l'image."
"Terraform"	"Infrastructure versionnée avec un état distant partagé (S3 + DynamoDB), modules communautaires épinglés à une version exacte, et code passé au crible de deux scanners de sécurité (tfsec, Checkov)."
"contrôles de sécurité"	"SAST, scan de secrets, scan de vulnérabilités conteneur, scan d'infrastructure — avec une politique claire sur ce qui bloque le pipeline (vulnérabilités corrigearables uniquement) et ce qui est documenté comme risque accepté."

Une phrase d'ouverture possible en entretien

"C'est le seul de mes trois projets que j'ai réellement déployé sur un vrai compte cloud plutôt que de le laisser purement théorique. Ça m'a forcé à découvrir et corriger deux vrais problèmes que je n'aurais jamais anticipés en me contentant d'écrire le code — je trouve que c'est ce qui distingue vraiment un projet DevOps 'qui marche sur le papier' d'un projet DevOps qui a vraiment tourné."

Limites du projet et pistes d'amélioration réelles

Limite	Ce qu'on ferait en vrai
Pas d'exposition publique (Ingress/Load Balancer)	Documentée et écrite (<code>k8s/examples/ingress-example.yaml</code>) mais non déployée, pour limiter le coût et la durée du test.
Un seul environnement testé en conditions réelles	L'overlay <code>prod</code> est écrit et validé statiquement (scan de sécurité, rendu Kustomize) mais jamais réellement déployé.
Pipeline GitLab non exécuté sur un vrai projet hébergé	Validé localement (syntaxe, exécution de plusieurs jobs via un simulateur) mais jamais lancé sur un vrai gitlab.com.
Scan ECR natif non concluant	<code>docker buildx</code> génère par défaut un index multi-plateforme incompatible avec le scan basique d'ECR (voir <code>reports/lessons_learned.md</code>) — contourné ici par un scan Trivy local, à corriger en désactivant la provenance (<code>--provenance=false</code>) pour un usage réel.
Un seul nœud, pas de haute disponibilité multi-zone	Choix volontaire de coût pour un test court ; une vraie production répartirait les nœuds sur plusieurs zones de disponibilité.

Questions probables en entretien + éléments de réponse

Q : "Racontez-moi un problème que vous avez rencontré et comment vous l'avez résolu."

R : Partie 10 — les deux bugs réels sont exactement faits pour cette question. Raconte le diagnostic (comment tu as compris la cause), pas seulement la solution.

Q : "Comment gérez-vous les coûts d'une infrastructure cloud ?"

R : Partie 11 — alerte de budget, instances Spot, destruction systématique, vérification indépendante après coup.

Q : "Comment sécurisez-vous un pipeline CI/CD ?"

R : Partie 7-8 — "shift left", plusieurs types de scans complémentaires, politique claire sur ce qui bloque vs ce qui est accepté.

Q : "Pourquoi Terraform plutôt que de cliquer dans la console AWS ?"

R : Partie 4.1 — reproductibilité, traçabilité, revue de code possible sur l'infrastructure elle-même.

Q : "Qu'est-ce qu'une NetworkPolicy et comment savez-vous qu'elle fonctionne ?"

R : Partie 6.3 et 10.1 — définition, puis le piège réel découvert, qui montre que tu sais qu'il faut tester activement, pas juste déclarer.

Q : "Quelle est la différence entre CI et CD ?"

R : Partie 3.5 — définition claire, avec l'exemple concret du `when: manual` pour la production dans ce projet.

Q : "Que feriez-vous différemment avec plus de temps ?"

R : Partie 14 — cite l'Ingress/Load Balancer et la haute disponibilité multi-zone en priorité.

Le laboratoire en direct : exposition publique et auto-réparation

Après la première vérification technique (Partie 9), une deuxième session de déploiement a été faite dans un but différent : **rendre le concept tangible**, en exposant l'application via un vrai lien public ouvrable dans un navigateur, et en démontrant en direct la capacité de Kubernetes à se réparer tout seul. Cette partie explique en détail chaque décision technique de cette session, pourquoi elle diffère de la première, et une troisième découverte réelle qui s'est produite en la démontant proprement.

16.1 Exposer l'application au monde entier : LoadBalancer vs ClusterIP vs Ingress

Rappel — les trois façons d'exposer un Service Kubernetes

ClusterIP (utilisé dans le déploiement de base, Partie 9) : une adresse IP valable uniquement à l'intérieur du cluster — invisible et injoignable depuis Internet. **LoadBalancer** : demande au fournisseur cloud (ici AWS) de créer un vrai répartiteur de charge public, avec une adresse joignable depuis Internet, dédié à ce seul Service. **Ingress** : un routeur HTTP plus évolué, capable de partager un seul point d'entrée public entre plusieurs applications selon le nom de domaine ou le chemin d'URL (voir `k8s/examples/ingress-example.yaml`, écrit mais non déployé dans ce projet).

Approche	Avantage	Inconvénient	Utilisée quand
ClusterIP	Gratuit, sécurisé par défaut (rien n'est exposé)	Inaccessible depuis l'extérieur	Communication interne entre services (le cas par défaut de ce projet)
LoadBalancer	Simple : une seule ligne de configuration, AWS gère tout le reste	Un répartiteur de charge par Service (coûteux si on a beaucoup de services), pas de routage par nom de domaine	Choisi ici : une seule application à exposer, pour une démonstration courte
Ingress	Un seul point d'entrée pour plusieurs	Demande d'installer et de configurer un contrôleur supplémentaire (AWS Load	Une vraie mise en production, avec plusieurs applications à

	applications, gestion fine du HTTPS	Balancer Controller), plus de pièces mobiles	exposer sous un même domaine
--	-------------------------------------	--	------------------------------

À retenir pour l'entretien

"J'ai choisi un Service de type LoadBalancer plutôt qu'un Ingress pour cette démonstration ponctuelle, en connaissance de cause : c'est la solution la plus simple quand on a une seule application à exposer rapidement, mais elle ne passe pas à l'échelle (un répartiteur de charge par application). Pour une vraie plateforme avec plusieurs services, j'aurais utilisé un Ingress avec l'AWS Load Balancer Controller, dont j'ai déjà écrit et documenté la configuration dans le dépôt sans la déployer, pour rester économe sur un test de quelques minutes."

16.2 Le piège oublié : la NetworkPolicy bloquait aussi le trafic public

Un détail de sécurité facile à oublier

La `NetworkPolicy` de base de ce projet (`demo-service-allow` , Partie 6.3) n'autorise le trafic entrant que depuis des *Pods* du même namespace (`podSelector: {}`). Une requête venant d'Internet via le LoadBalancer n'est justement **pas** émise par un pod du cluster — sans règle supplémentaire, elle aurait été bloquée exactement comme le pod de test inter-namespace de la Partie 10.1, alors même que le Service était bien de type `LoadBalancer` et techniquement joignable au niveau réseau AWS.

Correction : une `NetworkPolicy` additionnelle, spécifique à l'overlay de démonstration (`k8s/overlays/dev/networkpolicy-public.yaml`), autorise explicitement le trafic entrant depuis n'importe quelle adresse IP (`ipBlock: 0.0.0.0/0`) mais **uniquement sur le port applicatif** (8000) — jamais une ouverture totale. C'est un exemple concret du principe "tout refuser par défaut, autoriser explicitement" appliqué avec précision : on élargit l'accès seulement là où c'est voulu, sans toucher au reste de la posture de sécurité du namespace.

16.3 L'auto-réparation : comment Kubernetes sait qu'il doit recréer un pod

Définition — Boucle de réconciliation (control loop)

Le cœur de Kubernetes n'est pas une suite d'instructions ("fais ceci, puis cela"), mais une **boucle sans fin** qui compare en permanence deux choses : l'état *désiré* (ce que le fichier de configuration décrit — "je veux 2 répliques") et l'état *réel* observé sur le cluster. Dès qu'un écart apparaît entre les deux, un composant du control plane (le "ReplicaSet controller") agit

immédiatement pour combler l'écart — sans qu'aucun humain n'ait à détecter le problème ni à déclencher une action corrective.

C'est très différent d'un script traditionnel qui s'exécuterait une fois puis s'arrêterait : cette boucle tourne **en continu**, toute la durée de vie du cluster, prête à réagir à n'importe quel écart, qu'il soit causé par une panne matérielle, une commande malveillante, ou simplement le hasard.

16.3.1 Ce qui a été observé, avec les vrais chronomètres

Moment	Observation réelle
T+0s	2 pods Running : ...5d9v5 et ...hmpf8
T+0s	Commande <code>kubectl delete pod ...5d9v5 --force --grace-period=0</code> exécutée
T+0s (immédiat)	<code>kubectl get pods</code> montre déjà un nouveau pod ...lpmv4, encore 0/1 (pas prêt)
~T+15s	...lpmv4 est 1/1 Running : entièrement opérationnel

Le nouveau pod apparaît **quasi instantanément** (avant même la fin de la commande de suppression) : le ReplicaSet controller a détecté la disparition dès que l'API Kubernetes a confirmé la suppression, et a immédiatement demandé la création d'un remplaçant — il n'attend pas un intervalle de vérification périodique, il réagit à l'évènement.

16.3.2 Pourquoi `--force --grace-period=0` plutôt qu'une suppression normale ?

Définition — Arrêt propre (graceful shutdown)

Par défaut, Kubernetes laisse à un pod supprimé un délai de grâce (30 secondes typiquement) pour terminer proprement ce qu'il est en train de faire (finir de répondre à une requête en cours, fermer une connexion à une base de données) avant de l'arrêter réellement — un signal `SIGTERM` poli est envoyé d'abord, un `SIGKILL` brutal seulement en dernier recours.

La commande utilisée dans cette démonstration (`--force --grace-period=0`) court-circuite volontairement cette politesse, pour simuler le pire scénario réaliste : un serveur qui s'éteint d'un coup, sans le moindre préavis (coupure électrique, crash noyau...). C'est un test plus exigeant qu'un arrêt propre — et le système s'en est très bien remis.

16.4 Ce que cette démonstration ne prouve PAS (honnêteté technique)

Les limites réelles de cette protection

Cette auto-réparation protège contre la perte d'un **pod**, tant qu'il reste au moins un nœud disponible pour en recréer un autre. Elle ne protège pas, telle quelle, contre : la perte du **nœud entier** (ce cas est en réalité aussi couvert par EKS via le "node group" qui remplace un nœud EC2 disparu, mais avec un délai plus long, de l'ordre de la minute) ; la perte de **données** si l'application stockait un état local sans volume persistant (notre application de démo est "stateless" — sans état à perdre, ce qui simplifie tout) ; ou une **erreur dans le code de l'application elle-même**, qui serait fidèlement recréée à l'identique dans chaque nouveau pod.

À retenir pour l'entretien

"Je sais expliquer non seulement ce que Kubernetes apporte, mais aussi ce qu'il n'apporte pas tout seul : l'auto-réparation gère la disponibilité face à des pannes d'infrastructure, pas les bugs applicatifs ni la perte de données d'un service avec état. Une candidature honnête sur ses limites vaut mieux qu'une promesse de solution magique à tout."

16.5 Troisième découverte réelle : le LoadBalancer n'est pas une ressource Terraform

Un piège de démantèlement découvert en détruisant l'infrastructure

Le Service Kubernetes de type `LoadBalancer` déclenche la création d'un vrai répartiteur de charge AWS (Network Load Balancer) — mais cette ressource n'existe nulle part dans le code Terraform : elle a été créée directement par un composant du cluster (le "cloud-controller-manager") qui parle à l'API AWS pour le compte de Kubernetes. Résultat : `terraform destroy` ne sait absolument pas qu'elle existe, et ne l'aurait **jamais** supprimée — un vrai risque de "ressource fantôme" qui continuerait à être facturée indéfiniment après une destruction pourtant réussie en apparence.

Ordre correct des opérations, appliqué dans ce projet :

1. `kubectl delete -k k8s/overlays/dev` — supprime le Service Kubernetes, ce qui déclenche automatiquement la suppression du vrai LoadBalancer AWS par le cloud-controller-manager.
2. **Vérification indépendante** auprès d'AWS (`aws elbv2 describe-load-balancers`) que le répartiteur de charge a bien disparu, avant de continuer.

3. `terraform destroy` — peut alors supprimer le VPC et ses sous-réseaux sans rencontrer de dépendance bloquante (un VPC ne peut pas être supprimé tant qu'une ressource, même non gérée par Terraform, y fait encore référence).

À retenir pour l'entretien — une règle générale précieuse

"J'ai appris une règle qui dépasse ce seul projet : tout ce que Kubernetes crée dynamiquement dans le cloud sous-jacent (LoadBalancers, volumes de stockage persistants...) échappe à l'état Terraform et doit être nettoyé *avant* de lancer un ``terraform destroy``, sous peine de laisser une ressource orpheline qui continue à coûter de l'argent silencieusement. Je vérifie maintenant systématiquement, après toute destruction, directement auprès du fournisseur cloud, plutôt que de faire confiance au seul message de succès de l'outil."

Glossaire complet de tous les termes techniques

Add-on (EKS)

Composant logiciel géré du cluster (ex: vpc-cni, coredns), dont AWS peut gérer la version et la configuration.

AWS (Amazon Web Services)

Plateforme de cloud computing d'Amazon.

Backend (Terraform)

L'endroit où Terraform stocke son fichier d'état — ici, un bucket S3 distant plutôt qu'un fichier local.

Boucle de réconciliation (control loop)

Mécanisme central de Kubernetes qui compare en continu l'état désiré (la configuration) à l'état réel du cluster, et corrige automatiquement tout écart entre les deux.

CI/CD

Intégration Continue / Déploiement Continu : automatisation des tests et de la mise en production à chaque changement de code.

Cloud-controller-manager

Composant du control plane Kubernetes qui traduit certaines ressources (ex: un Service de type LoadBalancer) en véritables ressources chez le fournisseur cloud sous-jacent — d'où le fait qu'un LoadBalancer AWS créé ainsi n'apparaît jamais dans l'état Terraform.

Cluster (Kubernetes)

L'ensemble formé par le control plane et les nœuds qui font tourner les applications.

CNI (Container Network Interface)

Le composant qui gère la mise en réseau des pods dans un cluster Kubernetes.

Conteneur

Environnement d'exécution isolé et reproductible embarquant une application et ses dépendances.

Deployment (Kubernetes)

Ressource qui décrit combien de répliques d'un pod doivent tourner et comment les mettre à jour.

DevOps / DevSecOps

Pratiques rapprochant développement et exploitation, la seconde intégrant la sécurité à chaque étape.

Docker

Outil de création et d'exécution de conteneurs.

ECR (Elastic Container Registry)

Service AWS de stockage d'images Docker.

EKS (Elastic Kubernetes Service)

Service AWS de cluster Kubernetes managé.

État Terraform (state)

Fichier qui fait correspondre le code Terraform aux ressources cloud réellement créées.

Grace period (délai de grâce)

Temps laissé à un pod pour s'arrêter proprement (SIGTERM) avant d'être tué de force (SIGKILL) ; 30 secondes par défaut, réductible à 0 pour simuler une panne brutale.

HPA (HorizontalPodAutoscaler)

Ajuste automatiquement le nombre de répliques d'un Deployment selon une métrique de charge.

IaC (Infrastructure as Code)

Décrire l'infrastructure dans des fichiers versionnés plutôt que de la configurer manuellement.

IAM (Identity and Access Management)

Service AWS de gestion des identités et des permissions.

Ingress (Kubernetes)

Ressource qui expose un service à l'extérieur du cluster, généralement via un load balancer.

Kubernetes

Système d'orchestration qui gère automatiquement des conteneurs sur plusieurs machines.

Kustomize

Outil qui permet de décliner une configuration Kubernetes de base en plusieurs variantes par environnement.

LoadBalancer (type de Service)

Type de Service Kubernetes qui demande au fournisseur cloud de créer un vrai répartiteur de charge public pour cette application.

Module (Terraform)

Bloc de code Terraform réutilisable, souvent partagé par la communauté (ex: module VPC).

Namespace (Kubernetes)

Espace de noms qui isole logiquement des ressources au sein d'un même cluster.

NAT Gateway

Composant réseau qui permet à des machines dans un sous-réseau privé d'accéder à Internet sans y être directement exposées.

NetworkPolicy

Règle qui restreint quels pods peuvent communiquer entre eux et sur quels ports.

Node (Kubernetes)

Une machine qui fait réellement tourner des pods.

Pod

La plus petite unité déployable dans Kubernetes, contenant un ou plusieurs conteneurs.

Pod Security Standard

Fonctionnalité native de Kubernetes qui peut refuser la création de pods ne respectant pas un niveau de sécurité donné.

ReplicaSet

Ressource Kubernetes (généralement gérée automatiquement par un Deployment) chargée de maintenir en permanence le nombre exact de répliques d'un pod souhaité.

SAST (Static Application Security Testing)

Analyse du code source à la recherche de failles, sans l'exécuter.

Service (Kubernetes)

Adresse réseau stable qui redirige vers les pods actuellement disponibles.

SIGTERM / SIGKILL

Deux signaux Unix pour arrêter un processus : SIGTERM demande poliment un arrêt propre, SIGKILL tue immédiatement sans possibilité de nettoyage.

Spot (instance)

Capacité de calcul AWS louée à prix réduit, avec le risque qu'elle soit récupérée par AWS avec un court préavis — adapté aux charges non-critiques.

Stateless (sans état)

Se dit d'une application qui ne conserve aucune donnée localement entre deux requêtes — la perdre et la recréer ne fait perdre aucune information.

Terraform

Outil d'infrastructure-as-code multi-fournisseurs cloud.

Trivy

Outil de scan de vulnérabilités pour les images de conteneurs et le code.

VPC (Virtual Private Cloud)

Réseau privé virtuel isolé au sein du cloud d'un fournisseur.